



Able to Optimize database performance through partitioning and indexing, Reduce latency with intelligent caching strategies, Design highly available database architectures, Leverage cloud-managed databases for scalability and reliability	Unit VII: Database Scalability and Optimization 7.1 Database Partitioning (Horizontal vs. Vertical) 7.2 Caching Strategies (Redis, Memcached) 7.3 Load Balancing and High Availability 7.4 Indexing Strategies (Local, Global, Secondary) 7.5 Monitoring and Performance Testing	6 hrs
--	--	--------------

Yuba Raj Devkota

NCIT

Subject: Database Engineering (MS/ME)

Database Partitioning (Horizontal vs. Vertical)

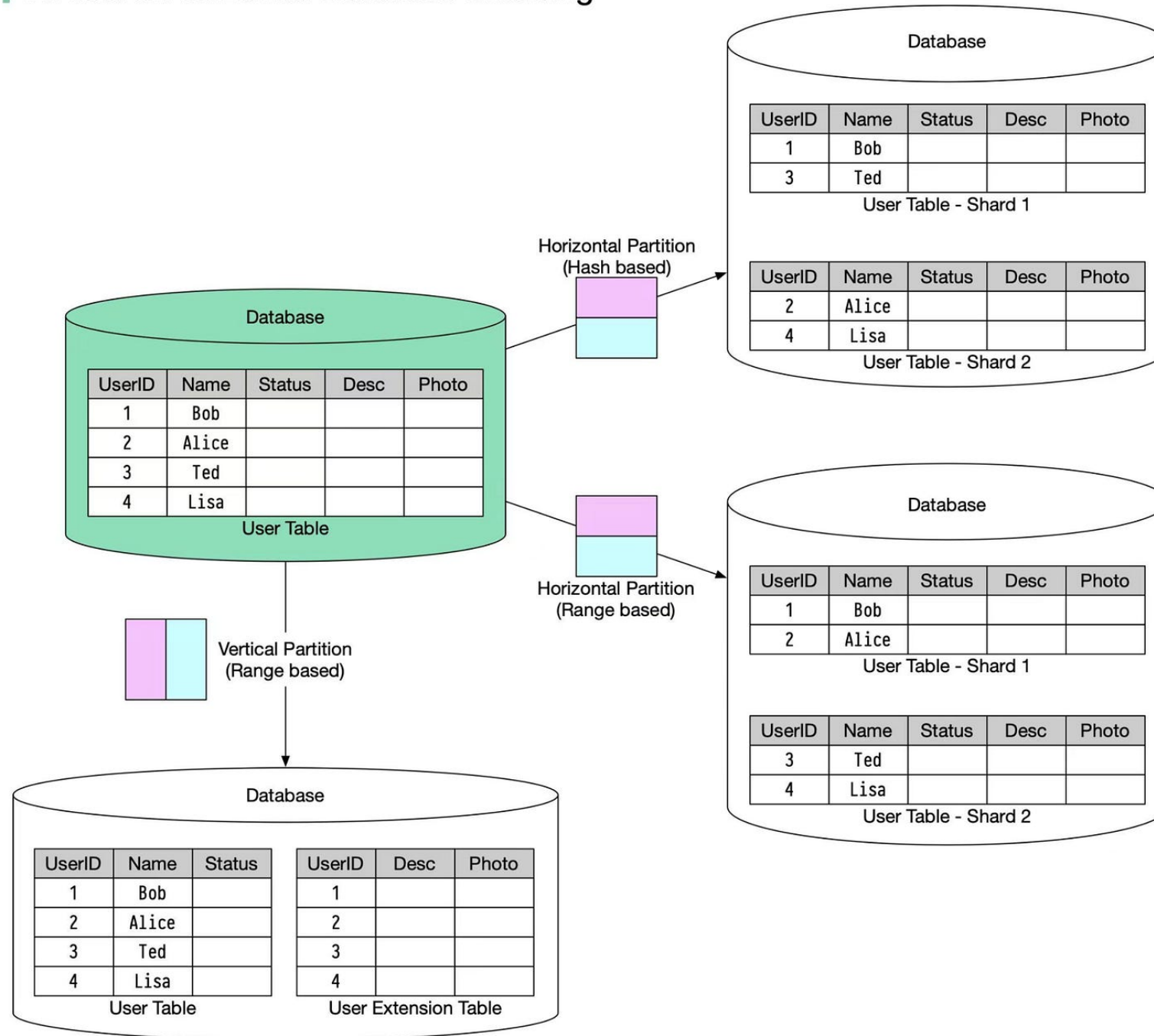
1. What is Database Partitioning?

Partitioning is a technique to divide a large database into smaller, more manageable segments (partitions) while maintaining logical data integrity. It improves performance, manageability, and availability.

In many large-scale applications, data is divided into partitions that can be accessed separately. There are two typical strategies for partitioning data.

- ◆ Vertical partitioning: it means some columns are moved to new tables. Each table contains the same number of rows but fewer columns (see diagram below).
- ◆ Horizontal partitioning (often called sharding): it divides a table into multiple smaller tables. Each table is a separate data store, and it contains the same number of columns, but fewer rows (see diagram below).

Vertical & Horizontal Database Sharding



Horizontal partitioning is widely used so let's take a closer look.

Routing algorithm

The routing algorithm decides which partition (shard) stores the data.

- ◆ Range-based sharding. This algorithm uses ordered columns, such as integers, longs, timestamps, to separate the rows. For example, the diagram below uses the User ID column for range partition: User IDs 1 and 2 are in shard 1, User IDs 3 and 4 are in shard 2.
- ◆ Hash-based sharding. This algorithm applies a hash function to one column or several columns to decide which row goes to which table. For example, the diagram below uses **User ID mod 2** as a hash function. User IDs 1 and 3 are in shard 1, User IDs 2 and 4 are in shard 2.

Example:

A `Customers` table with 10 million rows can be split by region:

- `Customers_East` (rows where `region = 'East'`)
- `Customers_West` (rows where `region = 'West'`)

SQL Example (PostgreSQL):

sql

-- Create a partitioned table

```
CREATE TABLE customers (  
    id INT,  
    name VARCHAR(100),  
    region VARCHAR(50)
```

```
PARTITION BY LIST (region);
```

-- Create partitions

```
CREATE TABLE customers_east PARTITION OF customers FOR VALUES IN ('East');
```

```
CREATE TABLE customers_west PARTITION OF customers FOR VALUES IN ('West');
```

B. Vertical Partitioning

- **Definition:** Splits a table **column-wise** into smaller tables with fewer columns but the same number of rows.
- **Use Case:** Useful when tables have many columns, and some are accessed more frequently than others.
- **Advantages:**
 - Reduces I/O for queries that need only a subset of columns.
 - Improves caching efficiency.
- **Disadvantages:**
 - Requires joins to reconstruct the original table.

Example:

A `Products` table with many columns (`product_id`, `name`, `price`, `description`, `manufacturer_details`) can be split into:

- `Products_Basic` (`product_id`, `name`, `price`)
- `Products_Details` (`product_id`, `description`, `manufacturer_details`)

SQL Example:

```
sql

-- Original table
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2),
  description TEXT,
  manufacturer_details JSONB);

-- Vertical partitioning
CREATE TABLE products_basic (
  product_id INT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2));
```

```
CREATE TABLE products_details (
  product_id INT PRIMARY KEY,
  description TEXT,
  manufacturer_details JSONB,
  FOREIGN KEY (product_id) REFERENCES products_basic(product_id));
```

3. When to Use Which Partitioning?

Criteria	Horizontal Partitioning	Vertical Partitioning
Data Size	Large row count	Large column count
Query Pattern	Filter by a key (e.g., date, region)	Access only a subset of columns
Scalability Need	Distributed databases (sharding)	Optimizing single-database performance

4. Partitioning Strategies

- **Range Partitioning:** Based on a range of values (e.g., dates).
- **List Partitioning:** Based on discrete values (e.g., regions).
- **Hash Partitioning:** Distributes data using a hash function (for even distribution).

- **Horizontal Partitioning** → Good for distributing large datasets.
- **Vertical Partitioning** → Good for optimizing column-heavy tables.

Database caching strategies

- **Cache:** A temporary storage layer (usually in-memory like Redis or Memcached) that holds a subset of database data.
- **Purpose:** To serve repeated requests faster than querying the database each time.
- **Goal:** Reduce database load and improve response times.

Database caching strategies are techniques used to store frequently accessed data in a faster storage layer (cache) so that applications can retrieve it quickly without repeatedly querying the main database. This improves performance, reduces database load, and decreases latency.

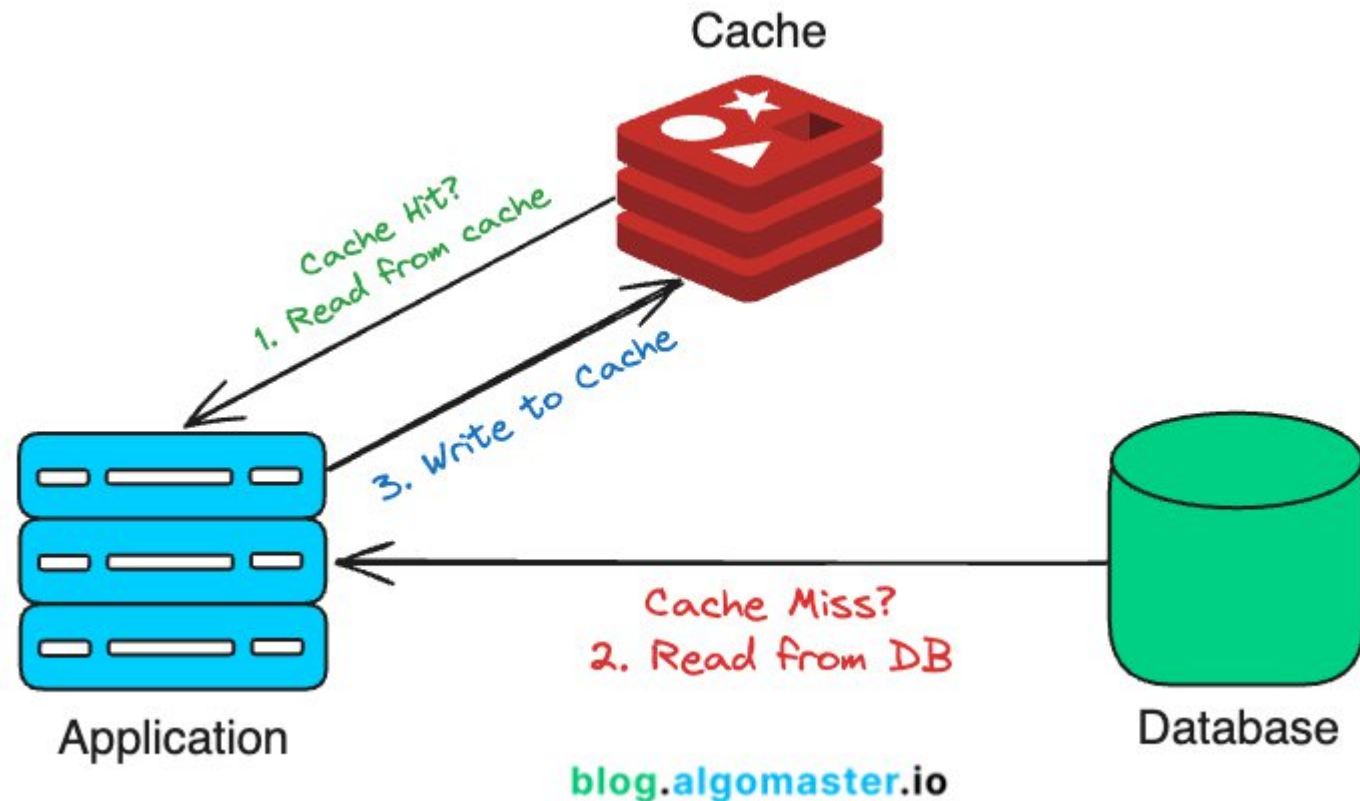
Common Database Caching Strategies

a) Cache-Aside (Lazy Loading)

- How it works:
 - The application first checks the cache for data.
 - If the data is in cache (**cache hit**), return it.
 - If not (**cache miss**), fetch from the database, store it in cache, then return it.
- Use case: Data that is read frequently but updated infrequently.

Scenario: User profile data

- - App requests user info for user_id = 101
- - Cache miss: Fetch from DB → Store in cache → Return data
- - Next request for user_id = 101: Cache hit → Return from cache

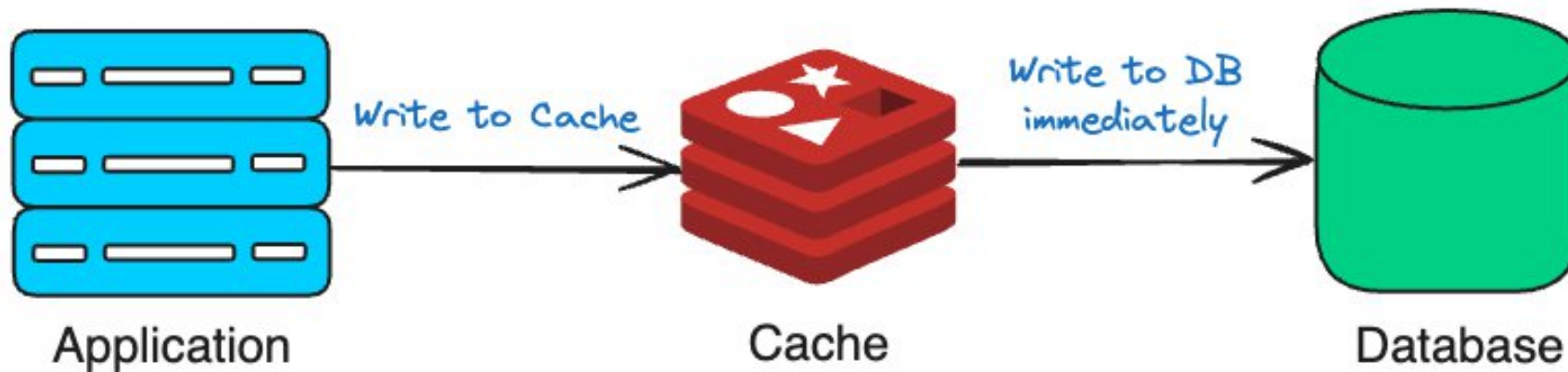


b) Write-Through Cache

- How it works:
 - Writes go to both the cache and the database simultaneously.
 - Ensures cache is always up-to-date.
- Use case: Systems that require strong consistency between cache and database.

Scenario: Updating product inventory

- - Update product quantity in cache → Simultaneously update DB
- - Next read will get latest data from cache

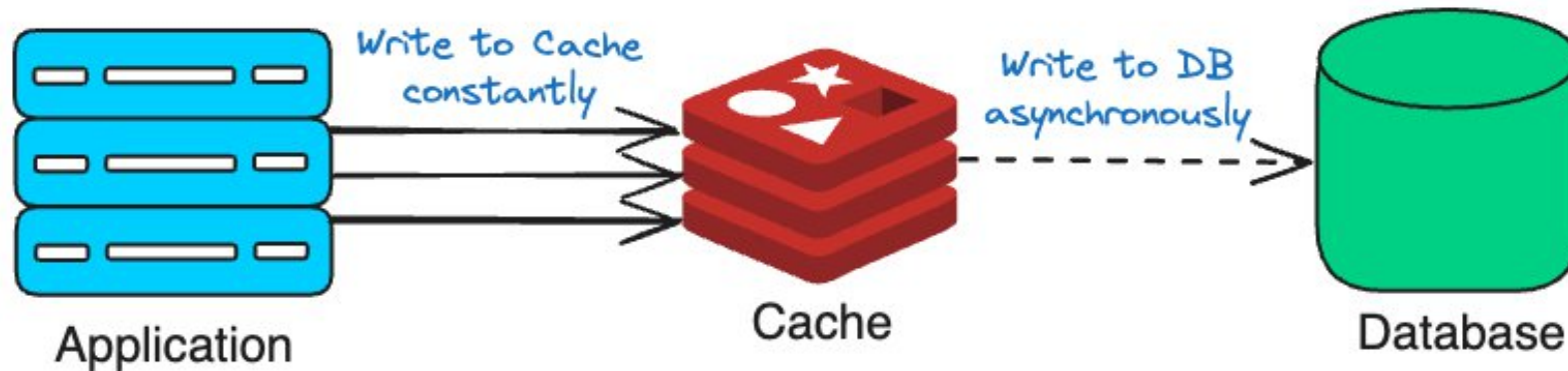


c) Write-Back (Write-Behind) Cache

- How it works:
 - Writes are done to the cache first.
 - Database update is delayed and done asynchronously.
- Use case: High-performance writes with eventual consistency.
- Risk: If cache fails before writing to DB, data may be lost.

Scenario: Logging user activity

- - Record activity in cache first
- - Periodically batch-write to DB



d) Read-Through Cache

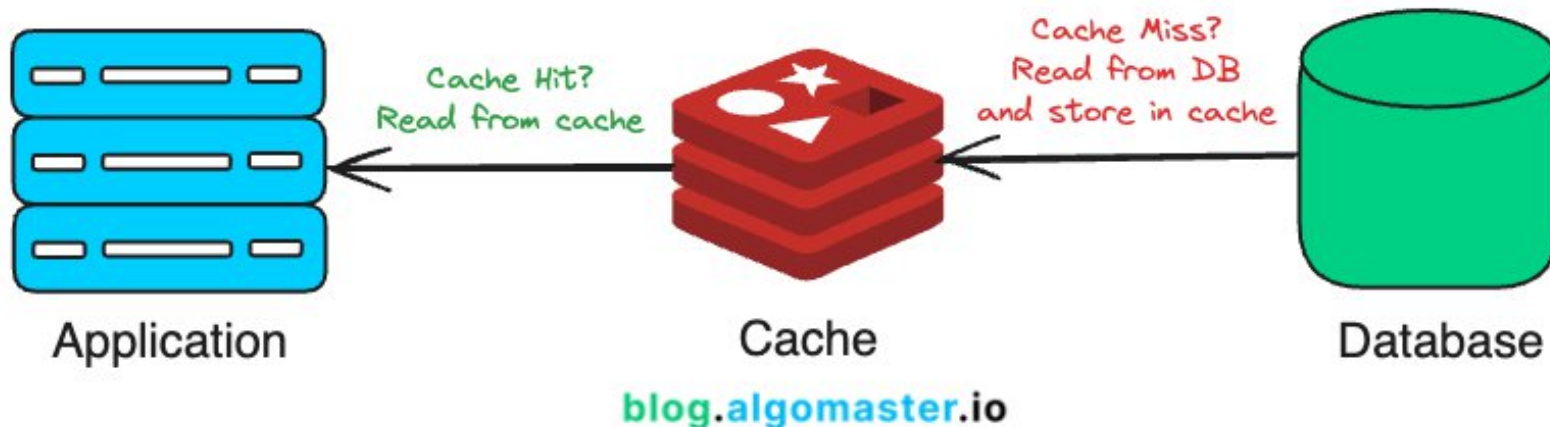
- **How it works:**
 - Application reads data from the cache only.
 - If cache misses, the cache itself fetches data from the database and returns it.
- **Use case:** Abstracts caching from the application logic.

Example:

text

Scenario: Fetching latest articles

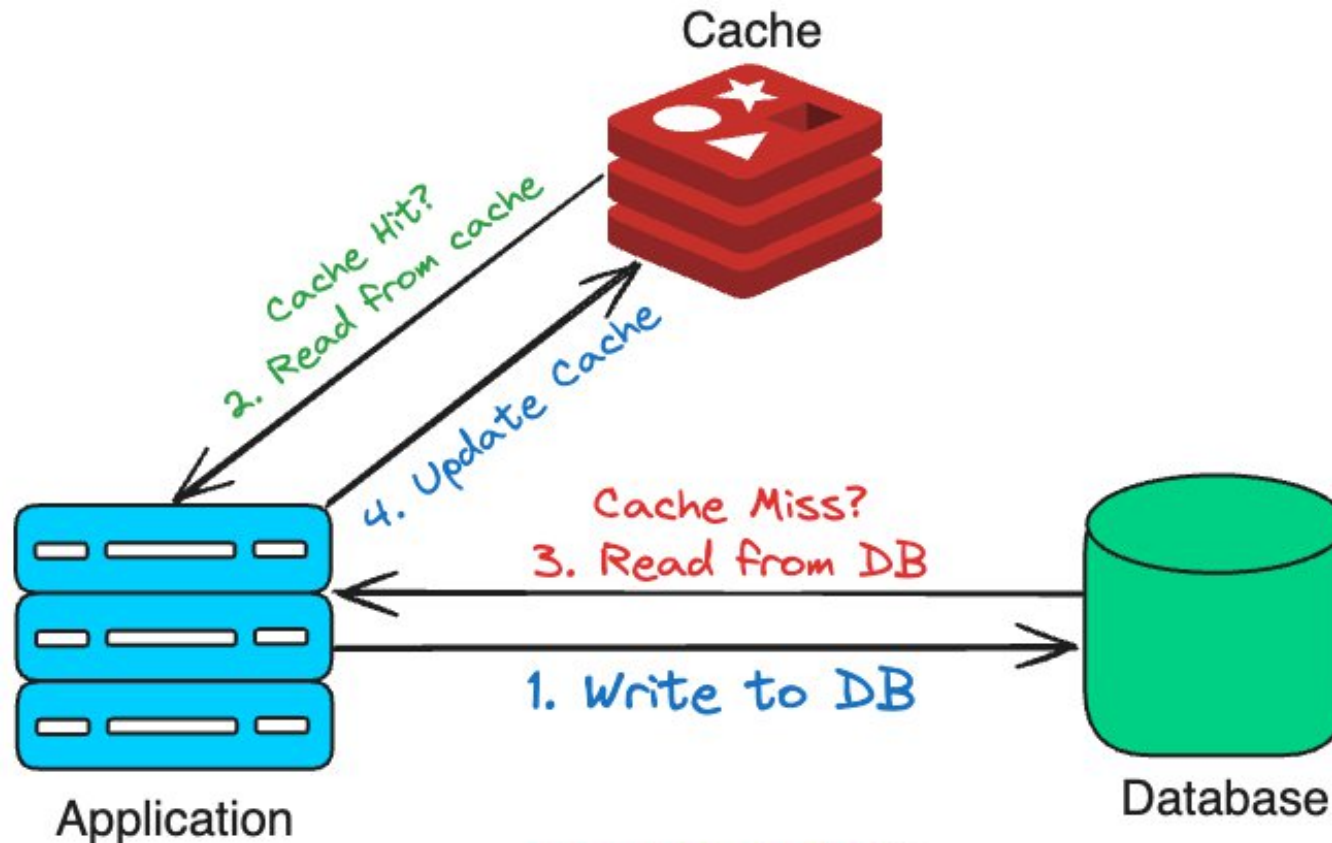
- App requests `article_id = 55` → Cache fetches from DB if missing → Returns to app



e. Write around Cache

Write Around is a caching strategy where data is written directly to the database, bypassing the cache.

The cache is only updated when the data is requested later during a read operation, at which point the **Cache Aside** strategy is used to load the data into the cache.



Scenario: E-commerce website showing product details.

1. User requests details of product ID=101 .
2. Check cache (Redis):
 - If found → return immediately (fast)
 - If not found → fetch from database → store in cache → return data
3. When the product price is updated:
 - If using **write-through**, update cache and DB immediately.
 - If using **write-back**, update cache now → DB updated later asynchronously.
4. Set TTL for cache → ensures data refreshes every 5 minutes.

✅ **Summary:**

- **Cache-Aside:** Lazy loading, app controls caching.
- **Write-Through:** Synchronous write to cache & DB.
- **Write-Back:** Async write to DB later.
- **Read-Through:** Cache automatically fetches on miss.
- **TTL/Expiration:** Controls data freshness.

Caching Strategies (Redis, Memcached)

1. What is Caching?

Caching stores frequently accessed data in fast-access memory (RAM) to reduce database load and improve performance.

1. What are Redis and Memcached?

Both **Redis** and **Memcached** are **in-memory key-value data stores** used to speed up applications by caching frequently accessed data, reducing database load, and improving response times.

- **Redis (Remote Dictionary Server)**

- An open-source, in-memory data structure store that supports multiple data types (strings, lists, sets, hashes, etc.).
- Offers persistence (can save data to disk) and advanced features like pub/sub, transactions, and Lua scripting.

- **Memcached**

- A high-performance, distributed memory caching system designed for simplicity and speed.
- Focuses on caching simple key-value pairs (strings) and is optimized for high throughput with minimal overhead.

1. Redis

- **Definition:**

Redis (Remote Dictionary Server) is an **open-source, in-memory data store** that can be used as a **database, cache, and message broker**.

- **Key Features:**

- Stores data **in-memory** for super-fast access.
- Supports **data structures** like strings, lists, sets, sorted sets, hashes, bitmaps, hyperloglogs, and streams.
- Supports **persistence**: data can be saved to disk (RDB or AOF).
- Supports **pub/sub messaging**.
- Provides **atomic operations** (like incrementing a value).

- **Use Cases:**

- Caching frequently accessed data
- Real-time analytics
- Session storage for web apps
- Leaderboards (gaming)
- Queues and pub/sub systems

- **Example:**

text

Scenario: Online shopping website

- Store user session info in Redis:

user:1234 -> {name: "John", cart_items: 3}

- Retrieve session data instantly without hitting main DB

2. Memcached

- **Definition:**

Memcached is a **high-performance, in-memory caching system** used to speed up dynamic web applications by alleviating database load.

- **Key Features:**

- Simple **key-value storage** (only strings, no complex data structures)
- **In-memory only** (no persistence to disk)
- Very fast and lightweight
- Does **not support replication or clustering natively** (although some client-side clustering is possible)

- **Use Cases:**

- Caching database query results
- Caching API responses
- Reducing repeated computations

- **Example:**

text

Scenario: Social media feed

- Cache latest posts for user_id = 101

key: "user:101:feed" -> value: [post1, post2, post3]

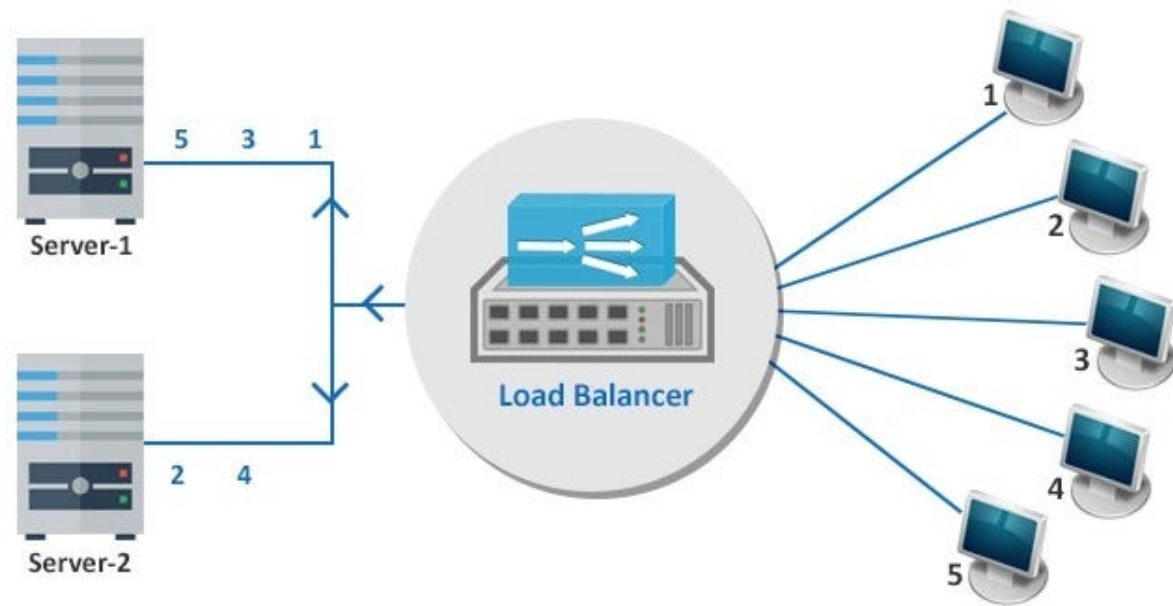
- Next time user requests feed → served from cache instantly

Feature	Redis	Memcached
Data Structures	Strings, Lists, Sets, Hashes, Sorted Sets, Bitmaps	Strings (simple key-value only)
Persistence	Yes (RDB & AOF)	No (memory only)
Memory Management	Can evict data, configurable	LRU eviction only
Replication	Yes (Master-Slave, Cluster)	No native support
Advanced Features	Pub/Sub, Lua scripting, transactions	None
Speed	Fast, slightly heavier due to features	Very fast, lightweight
Use Case	Caching + DB-like features + messaging	Pure caching, simple use cases

- Use Redis if:
 - You need **data persistence**.
 - You want **complex data structures**.
 - You need **pub/sub** or message queue features.
 - You want **atomic operations**.
- Use Memcached if:
 - You only need a **simple, in-memory cache**.
 - High speed with minimal memory overhead is critical.
 - You don't need persistence or complex data types.

✓ Summary:

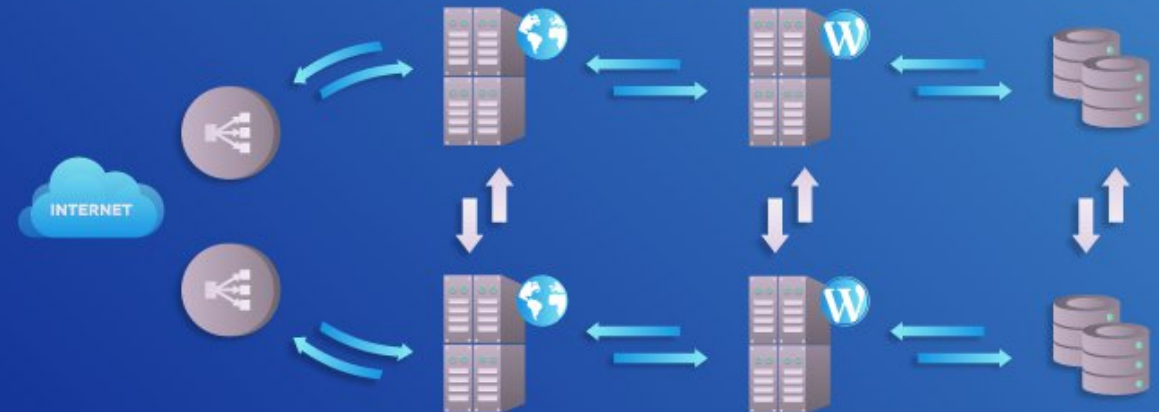
- **Redis:** Advanced, persistent, supports data structures & messaging.
- **Memcached:** Simple, ephemeral, pure key-value caching.



Load Balancing and High Availability

HIGH AVAILABILITY

N-TIER ARCHITECTURE



Load Balancing & Fault Tolerance



What is Load Balancing?

Load Balancing is the technique of distributing incoming requests (queries, transactions, user traffic) across multiple servers or database instances so that no single server is overloaded.

☞ *In DBMS, load balancing ensures:*

- *Faster query response*
- *Better scalability*
- *High availability*
- *Fault tolerance*

Load balancing is a process of distributing database workload evenly across multiple servers to improve performance, reliability, and scalability.

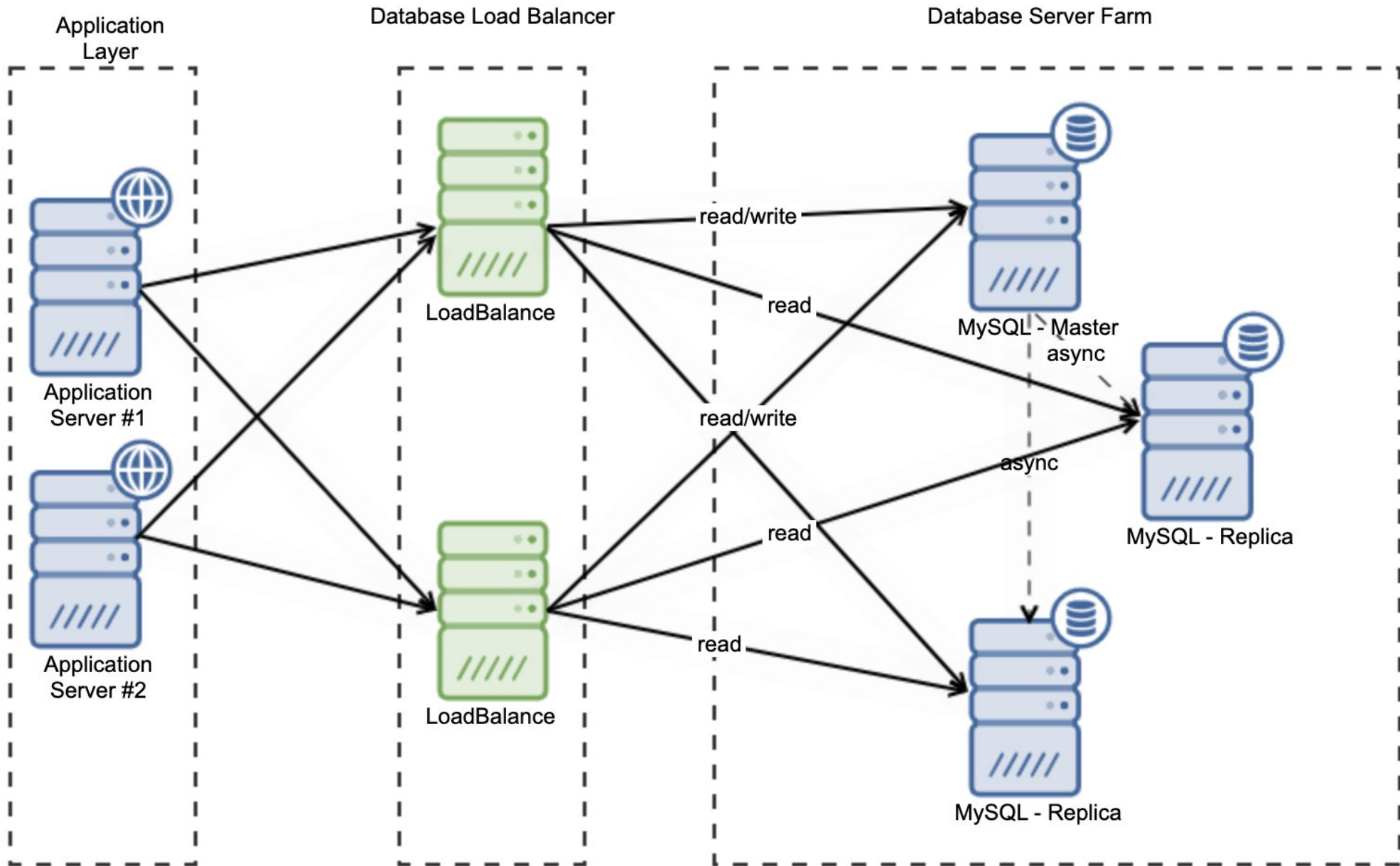
◆ Why Load Balancing is Needed in Databases?

Without load balancing:

- One database server becomes a **bottleneck**
- Slow query execution
- Server crash during peak load
- Poor user experience

With load balancing:

- Requests are shared
- System handles more users
- Easy to scale horizontally (add more servers)



In this load-balancing architecture, the database system is divided into **one Master database** and **multiple Slave (Replica) databases**.

- **Write operations** such as `INSERT` , `UPDATE` , and `DELETE` are handled by the **Master database**.
- **Read operations** such as `SELECT` queries are distributed among the **Slave databases** by the load balancer.
- The **Master replicates data** to all Slave databases to keep them synchronized.
- This reduces the load on the Master and improves **performance, scalability, and availability**.

👉 This technique is widely used in **high-traffic applications** like e-commerce and LMS portals.

- Load balancer sits between **application server and database**
- Distributes queries based on logic

Example:

- Read queries → replica servers
- Write queries → master server

Load Balancing Algorithms

A load balancing algorithm decides which server should handle a request when multiple database servers are available.

1 Round Robin Algorithm

How it works:

- Requests are distributed **one after another** to each server in order.
- No checking of server load.

Example:

If there are 3 database servers:

nginx

Request 1 → DB1

Request 2 → DB2

Request 3 → DB3

Request 4 → DB1

Use case:

- When all servers have **similar capacity**

Advantage: Simple to implement

Limitation: Not efficient if one server is slower

2 Least Connections Algorithm

How it works:

- Request is sent to the server with **least number of active connections**.

Example:

nginx

DB1 → 10 connections

DB2 → 4 connections

DB3 → 7 connections

New request → DB2

Use case:

- When request processing time varies

Advantage: Better performance

Limitation: Slightly complex

3 Weighted Load Balancing

How it works:

- Servers are assigned **weights** based on their power.

Example:

SCSS

DB1 (High power) → Weight 3

DB2 (Low power) → Weight 1

DB1 gets **3 requests** for every **1 request** to DB2.

Use case:

- Different server configurations

4 IP Hash Algorithm

How it works:

- Client IP address decides which server handles the request.
- Same client → same server

Example:

nginx

Client IP: 192.168.1.10 → DB2 (always)

Use case:

- Session-based applications

Advantage: Session consistency

Limitation: Uneven load possible

5 Random Algorithm

How it works:

- Server is chosen randomly.

Example:

- Request → DB3 (randomly selected)

Use case:

- Small systems or testing

High Availability in distributed System

High Availability means that a **distributed system** remains **continuously operational** and accessible **even if one or more components fail**.

👉 In simple words:

High Availability ensures that the system is **always available with minimal downtime**.

High Availability is the ability of a distributed system to provide uninterrupted service by using redundant components and failover mechanisms.

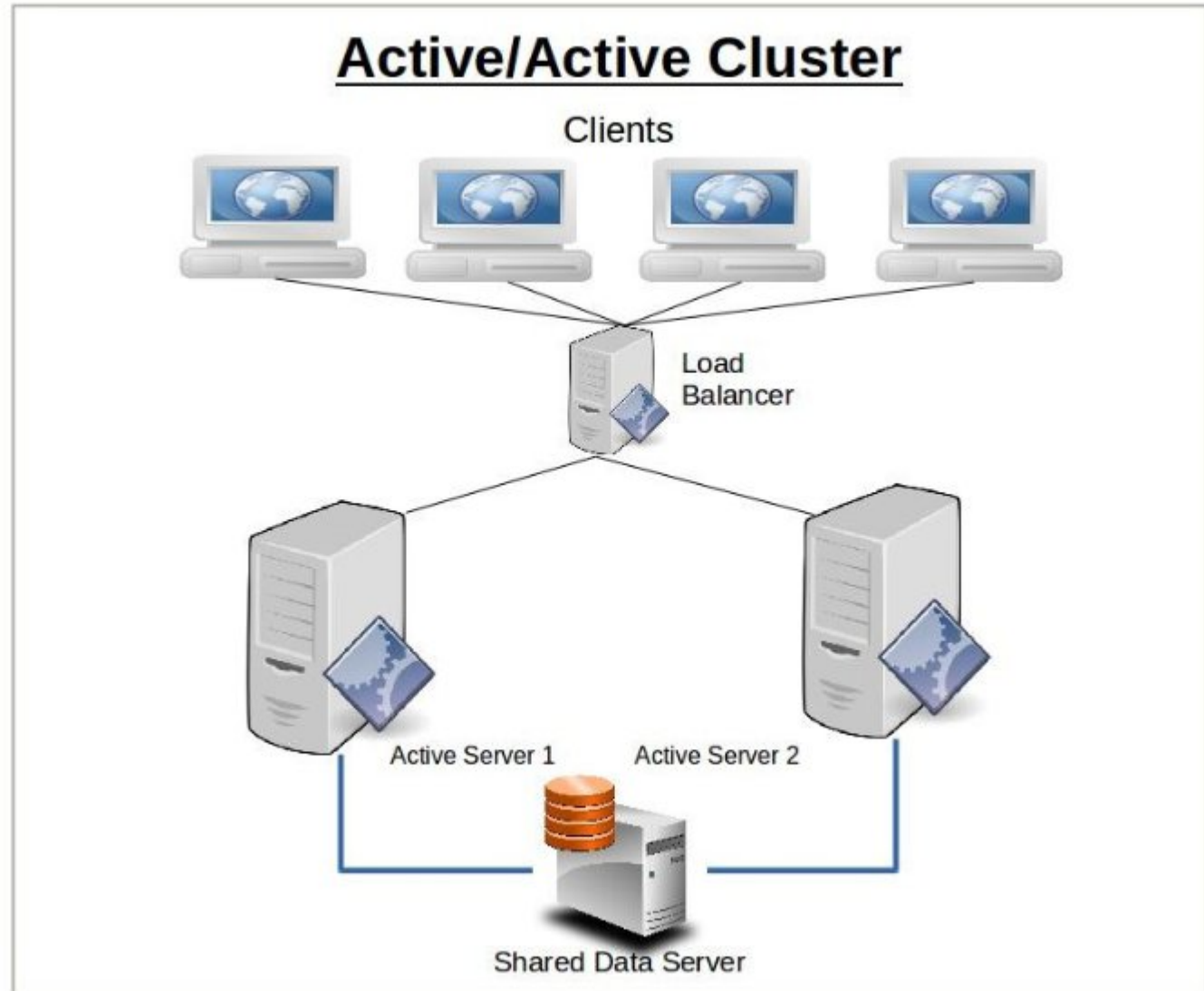
How High Availability Works

In a highly available system:

- Multiple servers (nodes) run **together**
- If one server fails, another **automatically takes over**
- Users experience **little or no service interruption**

This is achieved using:

- Redundancy
- Failover
- Load balancing
- Replication



1 Redundancy

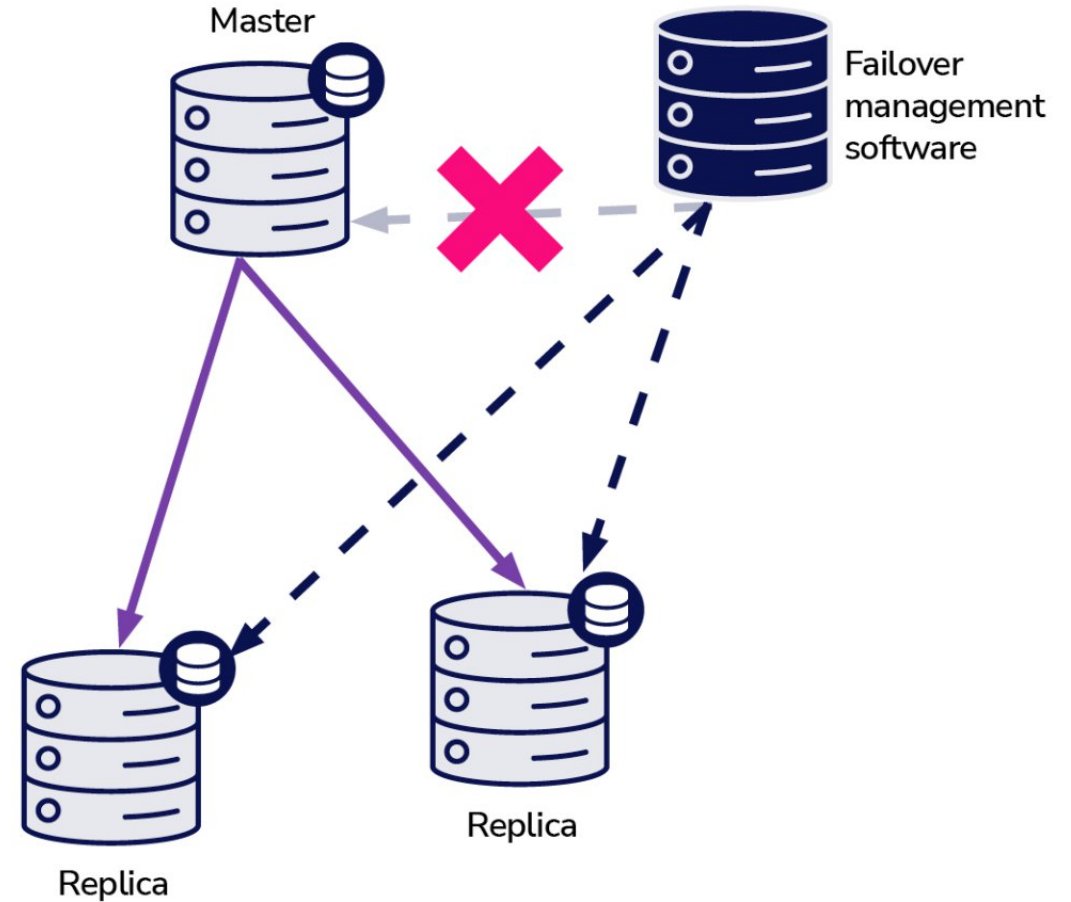
- Duplicate servers, databases, and network components
- No single point of failure

Example:

Two database servers instead of one

2 Failover Mechanism

- When the **primary server fails**
 - A **secondary (backup) server** automatically becomes active
- ✓ Ensures continuous service



3 Replication

- Data is copied across multiple nodes
- Ensures data availability even during failure

Example:

Master-Slave database replication

◆ Simple Example (Real Life)

🌐 Online Banking System

- If one server goes down during transaction
- Backup server immediately handles requests
- User does not notice the failure

4 Load Balancing

- Traffic is distributed among multiple servers
- Prevents overload and downtime

🎓 University / LMS System

- During exams or result publication
- If one server crashes
- Another server serves students without interruption

MySQL High Availability Example: Group Replication + Orchestrator

Orchestrator helps achieve high availability by monitoring database replication and automatically performing failover when the master database fails. It also supports load balancing by identifying healthy replicas and ensuring read queries are routed only to active database nodes.

Orchestrator is a database topology management and failover tool (commonly used with MySQL replication).

◆ Role in High Availability

- Orchestrator **continuously monitors** master and replica databases
- If the **master database fails**:
 - Orchestrator **automatically promotes a replica** as the new master
 - Updates the database topology
- This process is called **automatic failover**

✓ Result: **Minimal downtime → High Availability**

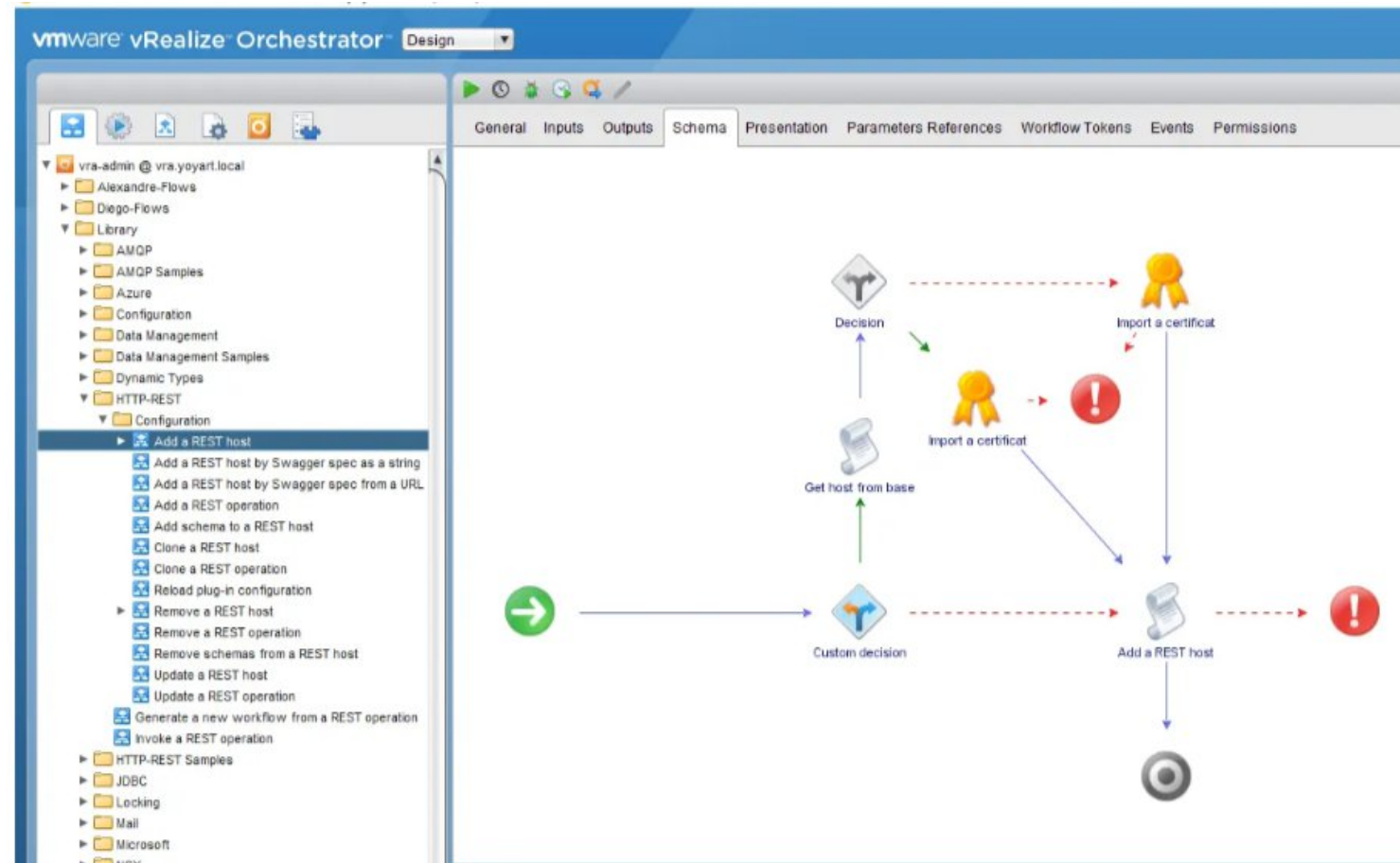
◆ Role in Load Balancing (Indirect)

- Orchestrator **knows** which nodes are master and replicas
- Helps applications or load balancers:
 - Send **WRITE queries** to the master
 - Send **READ queries** to healthy replicas
- Removes failed replicas from service

✓ Result: **Efficient read load distribution**

◆ Simple Example

- Master DB crashes ✗
- Orchestrator promotes Slave1 → Master ✓
- Load balancer redirects traffic automatically
- Users experience **no interruption**



Indexing Strategies

- *Indexing strategies are planned ways of creating and using indexes so that a database can retrieve data faster, especially when data size grows (scalability).*
- *An index works like a book index — instead of scanning every row (page by page), the DBMS jumps directly to the required data.*

◆ Why Indexing is Important for Scalability

- 🚀 Faster **SELECT / SEARCH** operations
- 📉 Reduced **disk I/O**
- 📊 Better performance for **large tables**
- ⚖️ Trade-off: slightly slower **INSERT, UPDATE, DELETE** due to index maintenance

Structure of an Index in Database



◆ Common Indexing Strategies

- Dense vs Sparse Indexing
- Single Column vs Multi-column (composite) indexing
- Clustered vs Non-clustered indexing (secondary indexing)
- Local and Global Indexing

i. Dense vs Sparse Indexing Strategies

A Dense Index is an index where every record (or every search key value) in the data file has an index entry.

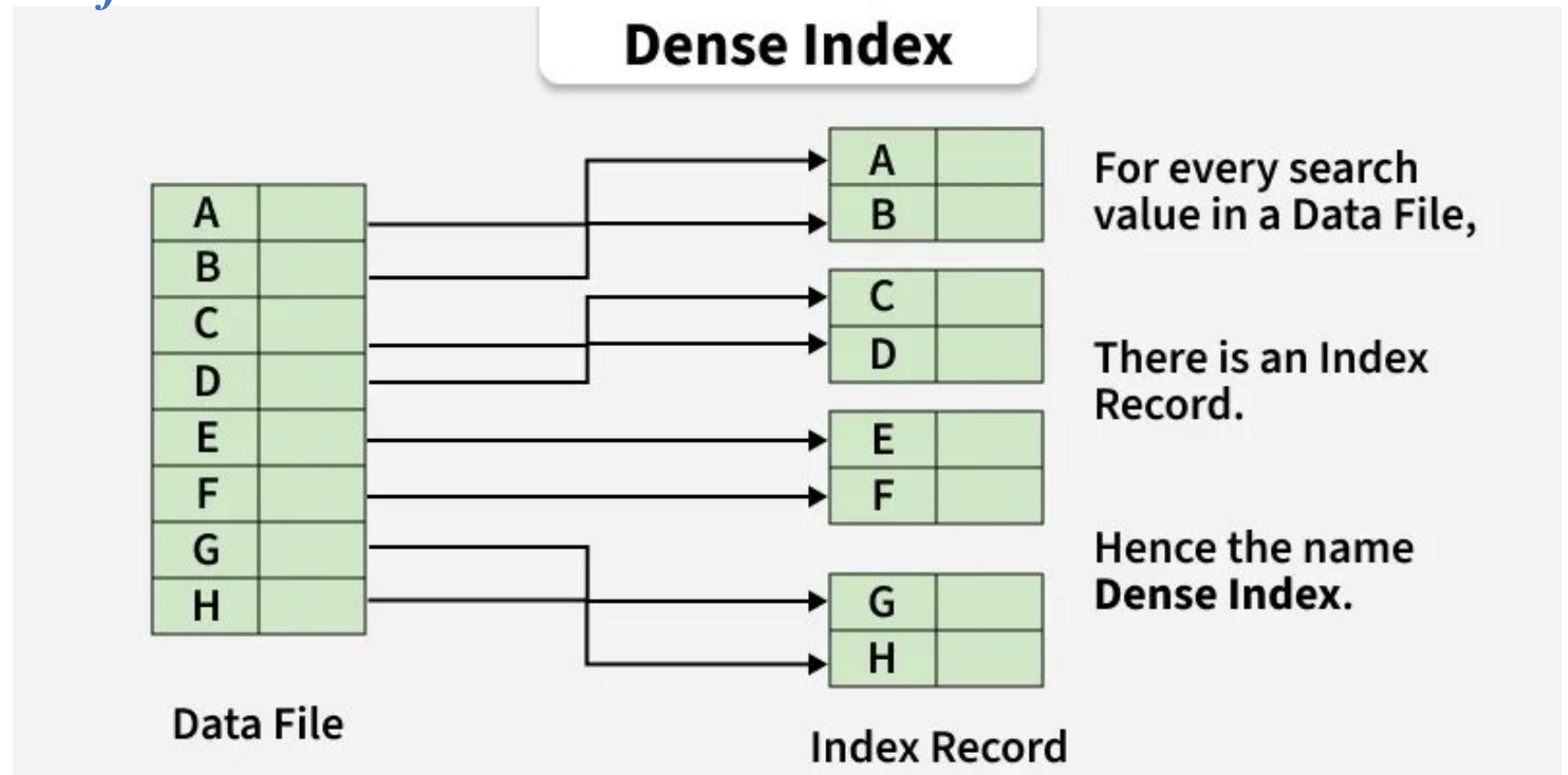
➡ *Index entry count $\approx$ number of records*

✓ Advantages

- Very fast search
- Direct access to records

✗ Disadvantages

- Needs more storage
- Slower updates (insert/delete)



A Sparse Index is an index where only some records (usually the first record of each block) have index entries.

🔑 Index entry count < number of records

✓ Advantages

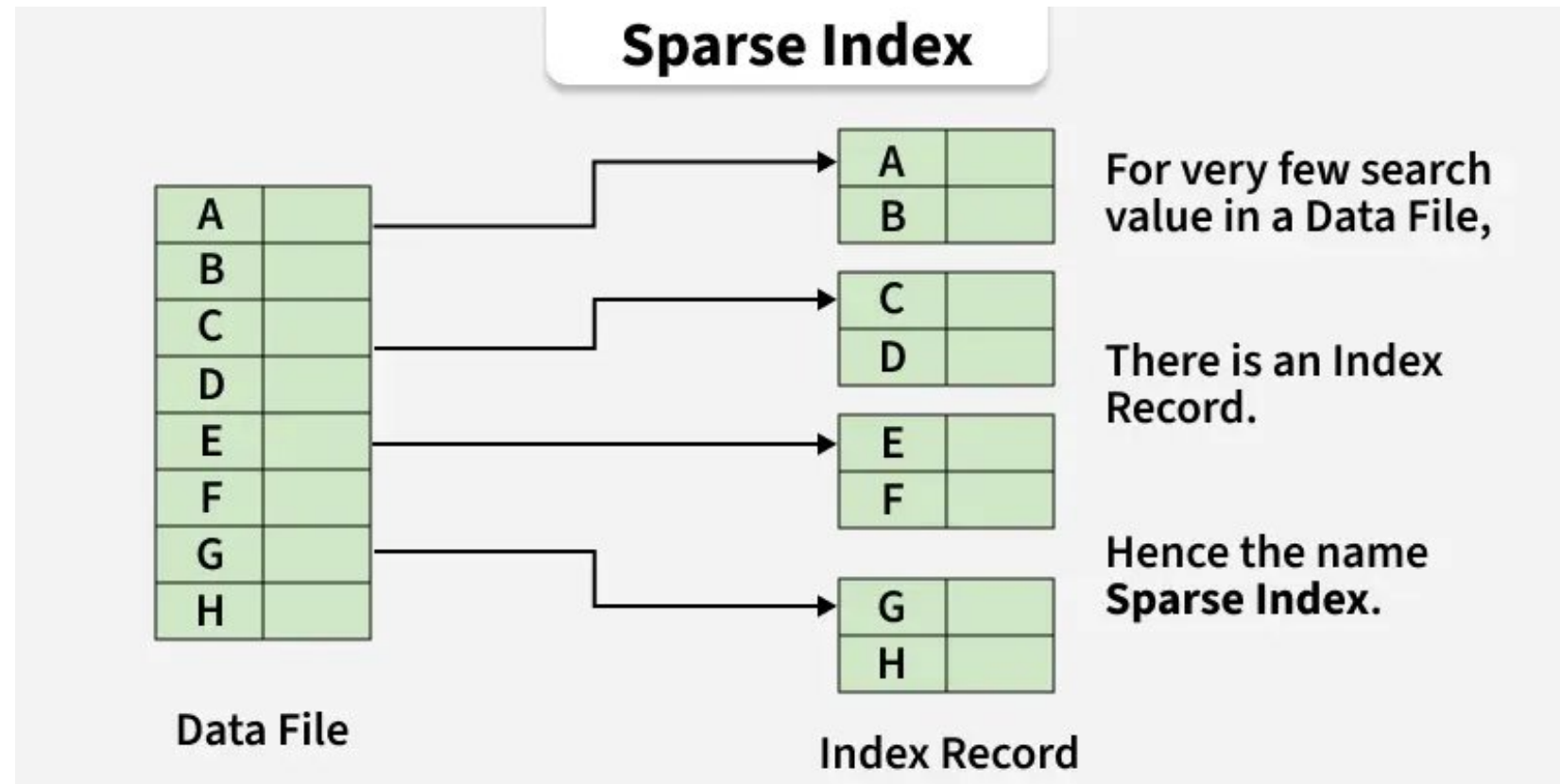
- Less storage required
- Faster index maintenance

✗ Disadvantages

- Slightly slower than dense index
- Requires **sorted data file**

📌 To find Roll No = 4:

1. DBMS goes to **Block 2**
2. Then searches inside the block



ii. Single Column vs Multi Column Indexing

Indexing strategies decide how many columns are used to build an index so that query performance improves as data grows.

1 Single-Column Indexing

A Single-Column Index is an index created on only one column of a table.

Table: Student

Roll_No	Name	Department
1	Ram	IT
2	Shyam	HR
3	Hari	IT

Index created on: Department

```
SELECT * FROM Student WHERE Department = 'IT';
```

✓ When to Use

- Queries frequently filter on **one column**
- Column has **high selectivity** (many unique values)

✓ Advantages

- Simple and small index
- Fast for one-column searches
- Low maintenance cost

✗ Limitation

- Less effective for queries using **multiple conditions**

➡ DBMS uses the **single-column index on Department** to quickly find matching rows.

2 Multi-Column Indexing (Composite Index)

A Multi-Column Index (Composite Index) is an index created on two or more columns together.

Table: Employee

EmpID	Department	Salary
1	IT	50000
2	IT	60000
3	HR	45000

Composite Index on: (Department, Salary)

```
SELECT * FROM Employee
WHERE Department = 'IT' AND Salary > 55000;
```

➡ DBMS efficiently uses the **multi-column index**.

✓ Advantages

- Faster multi-condition queries
- Reduces need for multiple indexes

✗ Disadvantages

- Larger index size
- Column order matters
- Higher maintenance cost

iii. Cluster vs Non Cluster Indexing (Secondary Indexing)

1 Clustered Index

A Clustered Index is an index where the actual data rows are stored in the same order as the index key.

👉 The index defines the physical order of the table.

⚠ Only ONE clustered index is possible per table.

Table: Student

Roll_No	Name
1	Ram
2	Shyam
3	Hari

- Clustered index on `Roll_No`
- Data is **physically stored in Roll_No order**

```
SELECT * FROM Student WHERE Roll_No = 2;
```

➡ DBMS directly finds the row because data is already ordered.

✓ Advantages

- Very fast range queries (`BETWEEN` , `<` , `>`)
- Efficient for sorted output

✗ Disadvantages

- Slow insert/update (data may need reordering)
- Only one clustered index allowed

2 Non-Clustered Index

A Non-Clustered Index is an index where index order and data order are different.

 *Index contains pointers to the actual data rows.*

 *Multiple non-clustered indexes can be created on a table.*

Table: Employee

EmpID	Name	Department
3	Hari	IT
1	Ram	HR
2	Shyam	IT

- Non-clustered index on Department
- Data remains **unsorted**
- Index stores → (Department → Row Address)

```
SELECT * FROM Employee WHERE Department = 'IT';
```

 DBMS uses index → jumps to matching rows.

Advantages

- Multiple indexes possible
- Faster search on non-key columns

Disadvantages

- Extra lookup step (index → data)
- Slightly slower than clustered index

iv. Local Indexing and Global Indexing

A Local Index is an index where each table partition has its own separate index.

1 Local Index

- *Index is aligned with table partitions*
- *Common in partitioned databases*
- *Used heavily in data warehouses*

Table: `Student_Marks` (partitioned by `Year`)

Partition	Data Stored
P1	Year = 2023
P2	Year = 2024

✓ Advantages

- Faster partition-level searches
- Easy maintenance
- Good for scalability

Each partition has its **own index** on `Roll_No` .

✚ Searching a student from **2024** → DBMS checks **only index of P2**, not others.

2 Global Index

A **Global Index** is a single index shared across all partitions of a table.

- Index is **not** partition-specific
- Covers **entire** table data

✓ Example

Table: `Orders` (partitioned by `Region`)

A global index on `Order_ID` points to rows in **all regions**.

📌 Searching by `Order_ID = 101`

→ DBMS uses **one global index**, regardless of partition.

✓ Advantages

- Very fast for **cross-partition queries**
- Useful when queries don't match partition key

✗ Disadvantage

- Costly to maintain when partitions change

Monitoring and Performance Testing

Monitoring and performance testing help us observe, measure, and improve database behavior so that systems remain fast, reliable, and scalable, especially under high load.

A. Database Monitoring

What is Database Monitoring?

Database monitoring is the **continuous tracking of database health and performance metrics** to detect issues like slow queries, high load, or failures **before users are affected**.

Key Metrics to Monitor

Metric	Meaning	Example
Query Response Time	Time taken to execute a query	<code>SELECT * FROM student;</code> taking 5 sec
CPU Usage	CPU consumed by DB processes	High CPU due to joins
Memory Usage	Buffer/cache usage	Cache miss issues
Disk I/O	Read/write speed	Slow HDD vs SSD
Connections	Active DB users	Too many connections causing delay
Locks & Deadlocks	Transaction conflicts	Two updates waiting forever

🔧 Monitoring Tools (with Examples)

Tool	Used For	Example
MySQL Performance Schema	Query & resource tracking	Detect slow SQL
EXPLAIN / EXPLAIN ANALYZE	Query execution plan	Find missing index
pg_stat_activity (PostgreSQL)	Active sessions	Find blocked queries
CloudWatch (AWS RDS)	Cloud DB monitoring	CPU, IOPS alerts
Prometheus + Grafana	Visual dashboards	Real-time charts

✅ Example (MySQL)

sql

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;
```

👉 Shows whether **index is used** or **full table scan** happens



MONITORING

CONFIGURATION

MY LINKS

MOST USED

Search

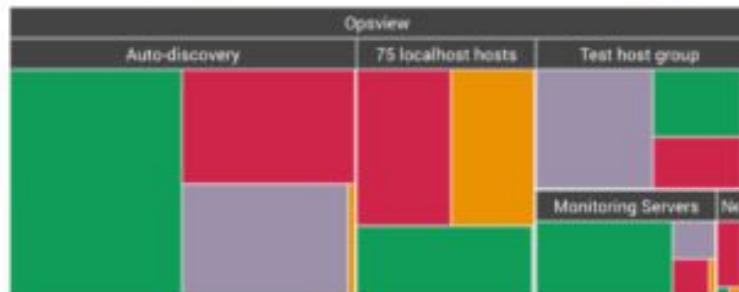


Administrator

CONFIGURATION

Initial Dashboard

HOST GROUP TREE MAP



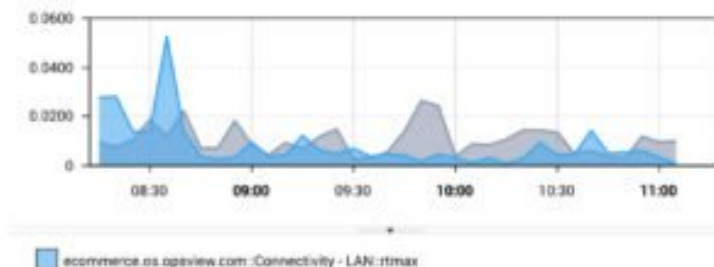
DISK SPACE



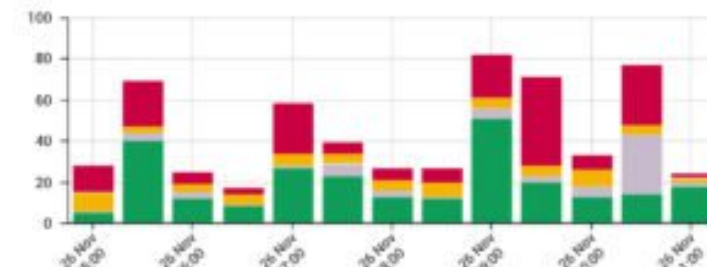
EVENT TABLE

TIME	STATUS	EVENT
Today 11:08	WARNING	PerfTest4467 :: AEF-QA-ServiceChe...
Today 11:00	WARNING	PerfTest4403 :: AEF-QA-ServiceChe...
Today 11:01	UP	image-builder.os.opsview.com
Today 11:05	UNKNOWN	PerfTest4428 :: AEF-QA-ServiceChe...
Today 11:01	UNKNOWN	PerfTest4470 :: AEF-QA-ServiceChe...
Today 11:11	OK	PerfTest4475 :: AEF-QA-ServiceChe...
Today 11:09	OK	PerfTest4467 :: AEF-QA-ServiceChe...

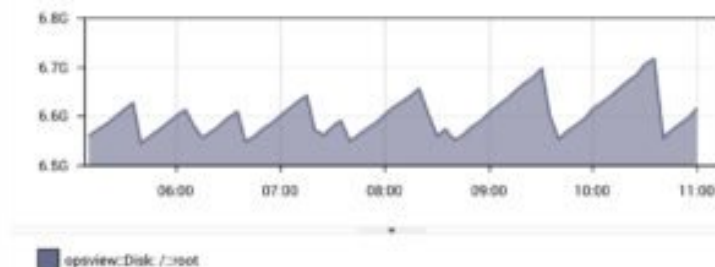
CONNECTIVITY



EVENT GRAPH



PERFORMANCE GRAPH



CPU STATS



PERFORMANCE GAUGE



BSM SERVICE



COMPONENT	AVAILABILITY	STATUS	OPERATIONAL DETAIL
GoldCollect...	0.00%	Failed	8 of 9 Host(s) failed
TestComp1	100.00%	Operational	All Hosts failed

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

- Startup / Shutdown
- Server Logs
- Options File

- Dashboard
- Performance Reports
- Performance Schema Setup

Information

No object selected

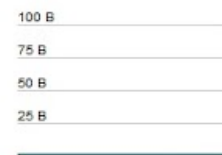
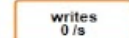
Statistics for network traffic sent and received by the MySQL Server over client connections.



Primary MySQL Server activity and performance statistics.



Overview of the InnoDB Buffer Pool and disk activity generated by the InnoDB storage engine.



 Action Output

#	Time	Action
---	------	--------

Message

Duration / Fetch

B. Database Performance Testing

Performance testing evaluates how well a database performs under different workloads, such as many users or large data volumes.

Types of Performance Testing

Type	Purpose	Example
Load Testing	Normal expected load	1,000 users
Stress Testing	Beyond capacity	10,000 users
Scalability Testing	Growth handling	Add nodes
Endurance Testing	Long-term stability	24-hour test

Performance Testing Tools

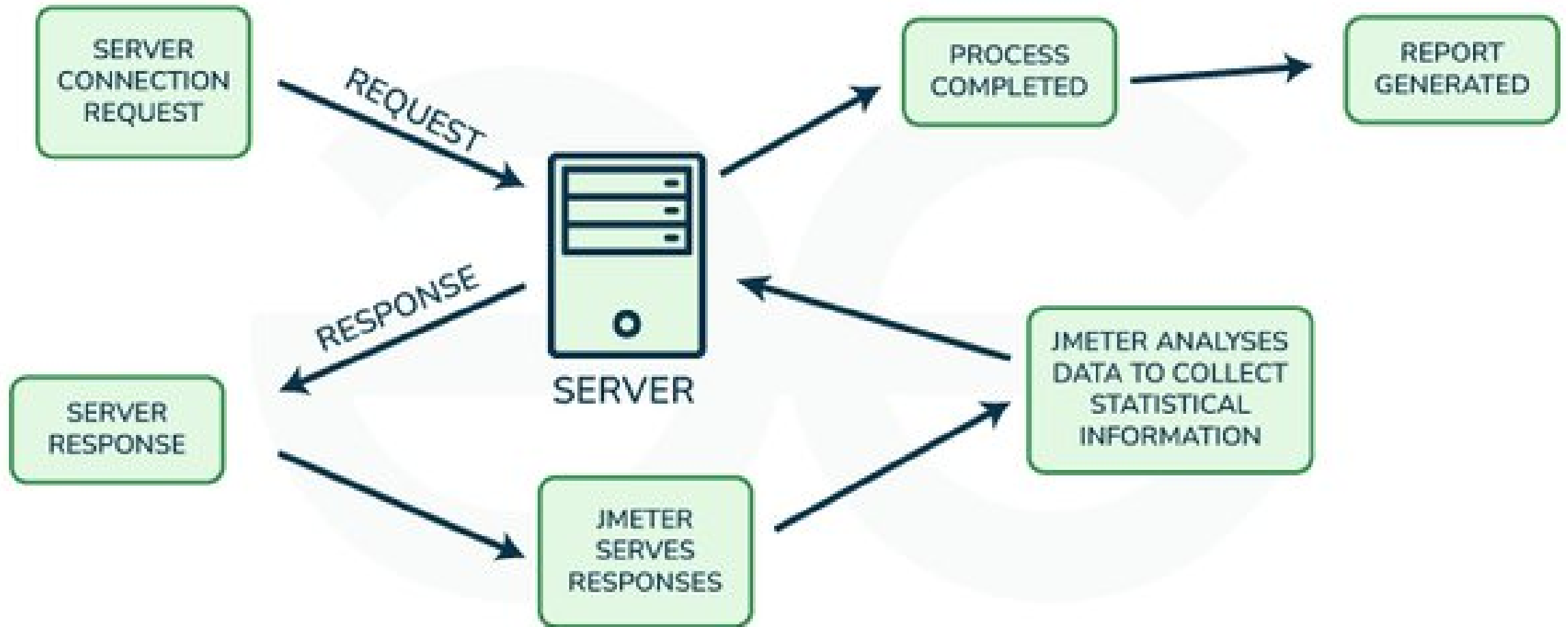
Tool	Database	Example
Apache JMeter	MySQL, PostgreSQL	Simulate 1,000 users
Sysbench	MySQL/PostgreSQL	OLTP benchmark
pgbench	PostgreSQL	TPS measurement
HammerDB	Enterprise DBs	TPC-C workload

Example (Sysbench – MySQL)

```
bash
```

```
sysbench oltp_read_write \  
--mysql-db=test \  
--mysql-user=root run
```

 Measures **transactions per second (TPS)** and latency



Monitoring	Performance Testing
Continuous	Periodic
Real-time	Simulated
Production-focused	Test environment
Detects problems	Predicts problems

Real-World Scenario



E-commerce Database

- Monitoring detects **slow checkout queries**
- EXPLAIN shows **missing index on order_id**
- Index added → response time reduced
- Load testing confirms system handles **5× more users**

Monitoring ensures database health in real time, while performance testing ensures the database can handle future growth and heavy load efficiently.

Homework

15. Differentiate between Redis and Memcached Caching Strategies. What are the various techniques for horizontal partitioning in databases? Explain any one of these techniques in detail. [PU, 2025]
16. Write short notes on: [PU, SOE]
- a. Local and Global Indexing
 - b. Blockchain database